

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Deemed to be University)

Patiala, Punjab

InvestCircle

A Peer-to-Peer Stock Research and Community Platform

PROTOTYPE STAGE REPORT

Submitted by

Ishan Jha	1024160099
Lakshay Sachdeva	1024160104
Aarav K. Arora	1024160106
Anirudh Phophalia	1024160120

Under the Guidance of

Anushka Ma'am

Department of Computer Science and Engineering
Thapar Institute of Engineering and Technology, Patiala
September 2026

Contents

1	Introduction	1
1.1	Background	1
1.2	Problem Statement	1
1.3	Objectives	2
1.4	Scope of This Report	2
2	Literature Survey / Existing Systems	3
2.1	Social Media and Discussion Forums	3
2.2	Financial News and Data Platforms	3
2.3	Portfolio and Watchlist Apps	3
2.4	Gap Identified	3
3	Requirement Analysis	4
3.1	Target Users	4
3.2	Scope	4
3.3	Functional Requirements	5
3.4	Non-Functional Requirements	5
3.5	Actors	6
3.6	Success Metrics	6
4	System Design	7
4.1	System Architecture	7
4.2	Deployment Diagram	8
4.3	Use Case Diagram	9
4.4	Class Diagram	10
4.5	Sequence Diagram	11
4.6	Activity Diagrams	11
4.7	Database Schema	15
4.8	API Design (Representative Endpoints)	15
5	UI/UX Design	16
5.1	Design Concept	16
5.2	Typography	16
5.3	Core Components	17
5.4	Screen Map	17

5.5	App Flow / Pipeline	18
5.6	Feature-to-Screen Mapping	19
6	Technology Stack	21
7	Work Completed So Far	22
7.1	Requirement and Design Documentation	22
7.2	System Design and Diagrams	22
7.3	Seed and Test Data	23
7.4	Real Historical Price Data	23
7.4.1	Data Provenance	24
7.4.2	Preprocessing Applied to the Real Dataset	24
7.5	Milestone Plan	25
8	Testing Strategy	26
8.1	Security Considerations	26
9	Conclusion and Future Work	27
9.1	Future Work	27
	References	28

List of Figures

4.1	High-level layered architecture of InvestCircle.	7
4.2	Deployment diagram — runtime nodes and their responsibilities.	8
4.3	Use case diagram for InvestCircle.	9
4.4	Class diagram — core domain model.	10
4.5	Sequence diagram — create post with a structured call.	11
4.6	Activity diagram — creation and automatic resolution of a structured stock call.	12
4.7	Activity diagram — publishing and interacting with a discussion.	13
4.8	Activity diagram — generating and consuming the personalized News Reel.	14
5.1	Screen map — navigation between core screens.	17
5.2	App flow — signup through call resolution and accuracy update.	18
5.3	Feature flowchart — screens and external API integration points.	20

List of Tables

3.1	Target user personas	4
3.2	Functional requirements	5
4.1	Core database tables	15
4.2	Representative REST API endpoints	15
5.1	Colour palette	16
5.2	Screen / feature breakdown	19
6.1	Technology stack	21
7.1	Milestone plan	25

Chapter 1

Introduction

1.1 Background

Retail participation in Indian equity markets has grown rapidly, and with it the volume of informal investment discussion on social media, forums, and dedicated "tip" channels. However, these channels share a structural weakness: opinions and predictions are published freely, but almost never tracked. A contributor who is right ten times and wrong ten times looks, on the surface, identical to one who has never been wrong — because neither platform keeps a record. Follower counts and engagement metrics end up substituting for the one signal that actually matters: whether a person's calls have historically been accurate.

InvestCircle is proposed as a direct response to this gap: a platform where every structured stock call a user publishes is tracked against real market outcomes and rolled into a public, permanent accuracy record, so that credibility is earned rather than claimed.

1.2 Problem Statement

Retail investors researching Indian equities currently combine financial news websites, social media discussions, and informal stock-tip communities, but these sources suffer from several concrete limitations:

- **Lack of accountability** — users can repeatedly publish predictions without any visible record of past outcomes.
- **Fragmented research** — news, discussion, and individual analysis are spread across unrelated platforms.
- **Unstructured opinions** — free-form text makes it difficult to identify an explicit, checkable prediction.
- **Weak credibility signals** — follower counts and engagement do not reflect historical prediction accuracy.
- **Limited personalisation** — users must manually search for news relevant to the stocks or sectors they actually care about.
- **Poor historical visibility** — unsuccessful predictions tend to disappear from view while successful ones remain prominent.

1.3 Objectives

- Give users a place to publish and discuss stock research with attached, trackable calls (e.g. "Buy TCS, target 12% in 3 months").
- Build a reputation system so accuracy over time is visible, not just follower count.
- Demonstrate solid full-stack software engineering practice: relational schema design, REST API design, authentication, caching, background job processing, and deployment.
- Keep AI features strictly secondary — isolated, optional enhancements, never a dependency for core flows.

1.4 Scope of This Report

This report is a **prototype-stage** report: it documents the requirement specification, system design, and UI/UX design completed so far, together with the supporting artefacts (seed/test data, AI-feature sample data, and real-price integration tooling) prepared to support development and the first working demonstration of the core call-tracking workflow. The same report template is used for the prototype-stage and final submissions, with this document reflecting the current, in-progress state of the project.

Chapter 2

Literature Survey / Existing Systems

2.1 Social Media and Discussion Forums

General-purpose platforms (Twitter/X threads, Reddit communities such as r/IndianStreetBets, Telegram "tip" groups) offer high engagement and low barrier to publishing, but no platform-level mechanism ties a prediction to its author's historical record. A user's credibility is inferred informally from tone and follower count rather than measured.

2.2 Financial News and Data Platforms

Platforms such as Moneycontrol, Economic Times Markets, and TradingView provide reliable market data and news, and TradingView in particular supports user-published trade ideas. However, these are either one-directional (news out, no structured community discussion in) or, where user ideas are supported, resolution of a published idea against its target is not a first-class, platform-tracked feature the way call resolution is central to InvestCircle.

2.3 Portfolio and Watchlist Apps

Watchlist-centric apps (broker apps, screener tools) let a user track tickers they personally care about, but are single-player by design — there is no social graph, no publishing of research, and therefore nothing to hold accountable.

2.4 Gap Identified

No reviewed platform combines all of: (a) identity-based, non-anonymous publishing, (b) a *structured*, machine-checkable prediction format (ticker, direction, target, horizon) rather than free text, (c) automatic resolution against real market data, and (d) a persistent, public accuracy record surfaced alongside a social graph and personalised content. This is the gap InvestCircle is designed to fill, and it is the basis for the functional requirements in Chapter 3.

Chapter 3

Requirement Analysis

3.1 Target Users

Table 3.1: Target user personas

Persona	Description	Primary Need
Active retail investor	Trades/invests regularly, wants to publish and track calls	Credibility, visibility, discussion
Passive learner	Reads research, doesn't post much	Discovery of reliable contributors, curated feed
Community moderator (stretch)	Keeps discussion quality high	Moderation tools, reporting

3.2 Scope

In Scope

- User accounts: signup/login, profile, bio, followed sectors
- Posts: research notes with an optional structured call (ticker, direction, target, horizon)
- Watchlists: create/edit public or private watchlists of tickers
- Follow/unfollow users; personalised feed of followed users' posts
- Comments and threaded replies on posts
- Accuracy tracking: automatic resolution of calls against price data, with a per-user track record
- Notifications: new follower, comment, call resolved
- Search: users, tickers, posts
- AI bolt-on: automatic sector and sentiment tagging of each post at creation time

Out of Scope (v1)

- Real brokerage integration or real-money trading
- Native mobile apps (web-responsive only)
- Paid subscriptions / monetisation
- Advanced recommendation engine beyond the follow-based feed

3.3 Functional Requirements

Table 3.2: Functional requirements

ID	Requirement	Priority
FR-1	Users can register, log in, and manage a profile	Must
FR-2	Users can create a post with an optional structured call (ticker, direction, target price, horizon date)	Must
FR-3	Users can create, edit, and share watchlists	Must
FR-4	Users can follow/unfollow other users	Must
FR-5	Feed shows posts from followed users, most recent first	Must
FR-6	Users can comment and reply on posts	Must
FR-7	System auto-resolves a call as Hit/Miss once its horizon date passes, using end-of-day price data	Must
FR-8	User profile shows an accuracy score and call history	Must
FR-9	Users receive in-app notifications for follows, comments, and resolved calls	Should
FR-10	Search across users, tickers, and posts	Should
FR-11	Each post is auto-tagged with sector and sentiment via an NLP model	Could (AI bolt-on)
FR-12	Admin/moderation: report and hide a post	Could

3.4 Non-Functional Requirements

- **Usability** — core flows (post, follow, comment) usable within 3 clicks from the feed.
- **Performance** — feed loads in under 1.5s for a user following up to 200 accounts (with pagination).
- **Reliability** — the call-resolution job must run daily without missing entries.
- **Security** — passwords hashed with bcrypt/Argon2, JWT-based session auth, input validation on all endpoints.
- **Scalability** — schema and API designed to support horizontal scaling of the API layer.
- **Maintainability** — documented REST API, seeded test data, CI running lint and tests on every PR.

3.5 Actors

- **User** — the primary actor; registers, publishes posts and calls, follows other users, manages watchlists, comments, searches, and views notifications and accuracy scores.
- **Administrator** — moderates content (reports/hides posts).
- **Call Resolution Scheduler** — an automated system actor that runs the daily job resolving expired calls.
- **AI Tagging Service** — an automated system actor that asynchronously tags posts with sector/sentiment.
- **Market Data Provider / News Provider** — external systems supplying price data and market news respectively.

3.6 Success Metrics

- System correctly resolves $\geq 95\%$ of due calls without manual intervention.
- Feed and profile pages meet the performance target under seeded load testing.
- All Must-priority functional requirements demonstrated end-to-end at final evaluation.
- Test coverage $\geq 70\%$ on the backend service/business-logic layer.

Chapter 4

System Design

4.1 System Architecture

InvestCircle follows a three-tier architecture: a React single-page frontend, a REST API backend with a business-services layer, and a PostgreSQL + Redis persistence layer, with background workers and the optional AI tagging microservice kept outside the request path so they can fail or lag without affecting core flows.

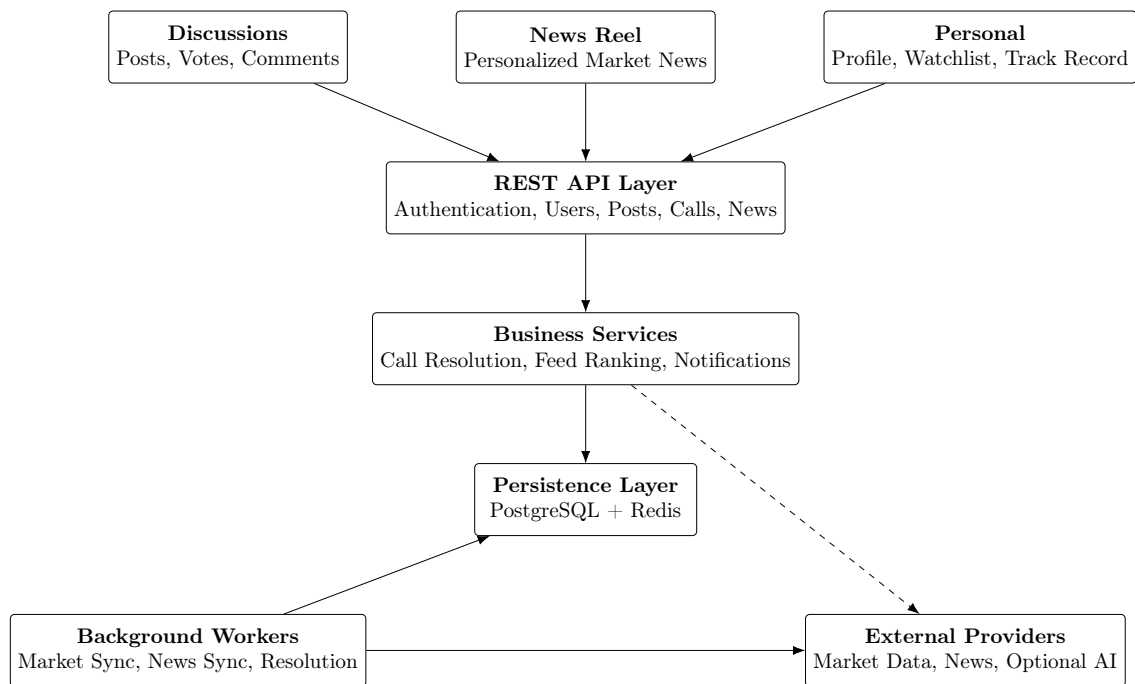


Figure 4.1: High-level layered architecture of InvestCircle.

4.2 Deployment Diagram

The deployment view maps the logical layers above onto physical/runtime nodes for the prototype deployment: a browser-based client, a containerised Node.js application server, a managed PostgreSQL instance, a Redis node for caching and background jobs, and the external services the app depends on.

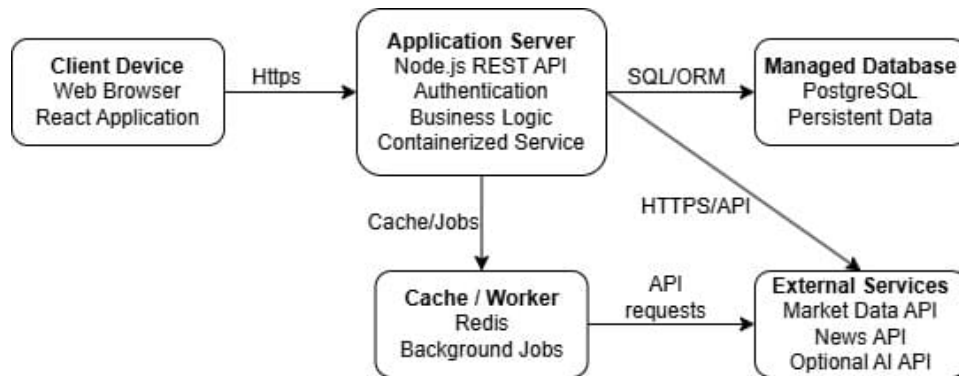


Figure 4.2: Deployment diagram — runtime nodes and their responsibilities.

4.3 Use Case Diagram

Figure 4.3 identifies the actors and use cases for InvestCircle, including the human actors (User, Administrator) and the external/automated actors the system integrates with (News Provider, Market Data Provider, AI Service). Use cases connected with `<<include>>` indicate a use case that always invokes another as part of its own flow (e.g. viewing personalised news always includes the vote/comment/reply interaction surface; viewing call history includes resolving expired calls).

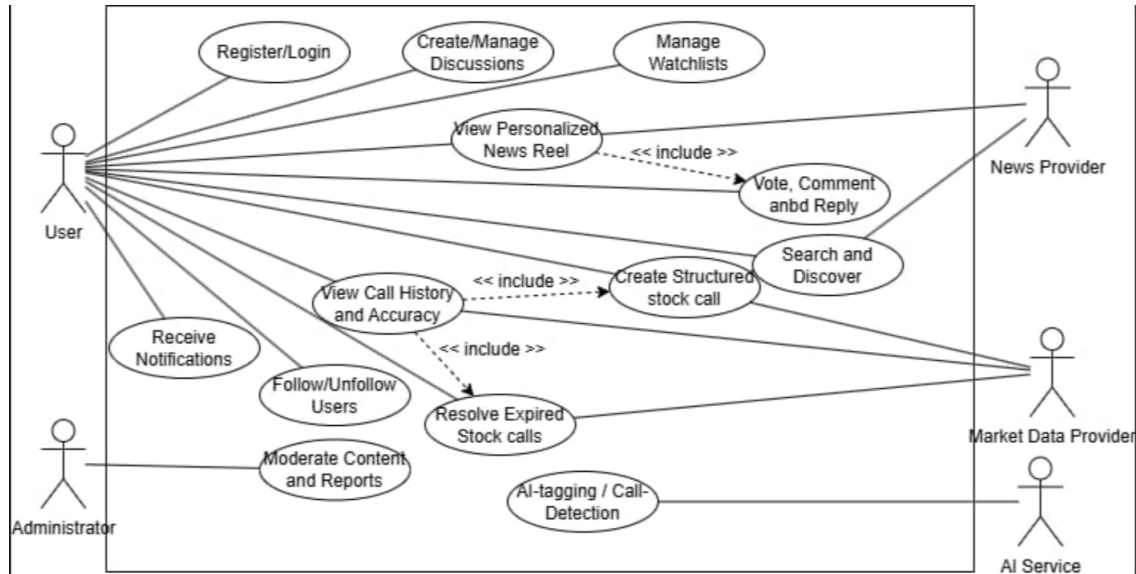


Figure 4.3: Use case diagram for InvestCircle.

4.4 Class Diagram

The class diagram (Figure 4.4) captures the core domain model underlying the relational schema described in Section 4.7: User, Post, Call, Comment, Follow, Watchlist, WatchlistItem, and Notification, along with their multiplicities.

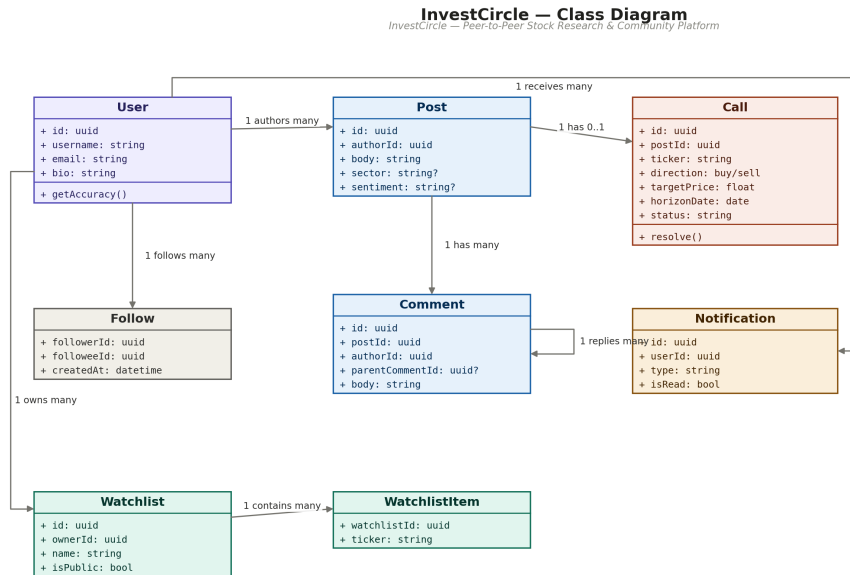


Figure 4.4: Class diagram — core domain model.

4.5 Sequence Diagram

Figure 4.5 traces the create-post-with-call flow end to end, highlighting that AI sector/sentiment tagging happens *asynchronously* via a Redis-backed queue: the user receives a response and sees their post in the feed immediately, while tagging happens in the background and is written back through an internal callback.

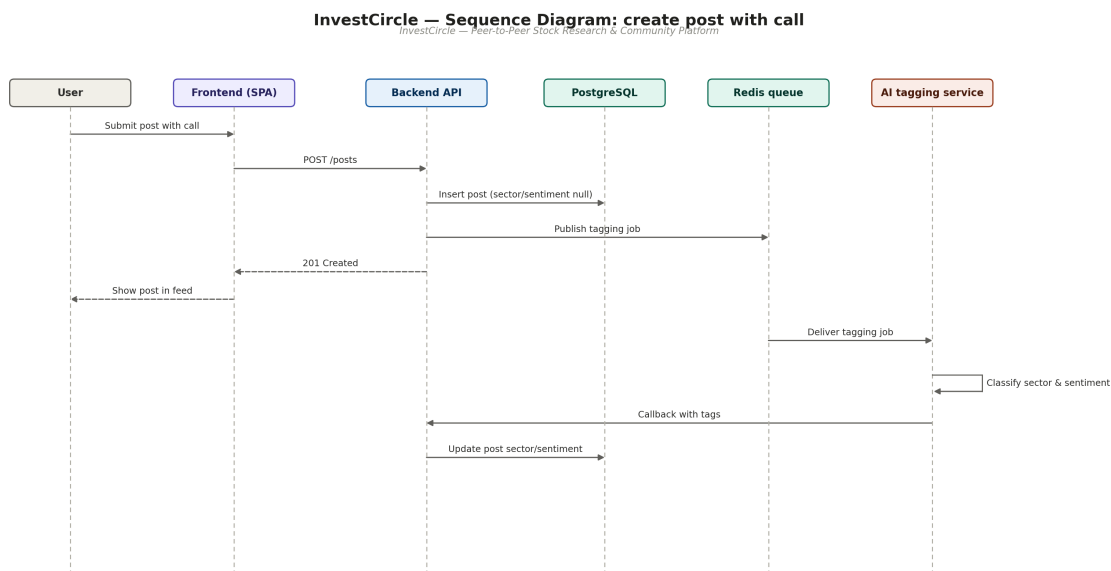


Figure 4.5: Sequence diagram — create post with a structured call.

4.6 Activity Diagrams

Three activity diagrams describe the platform’s core workflows in sequence: creating and resolving a structured stock call (the platform’s key differentiator), publishing and interacting with a discussion, and generating the personalised News Reel.

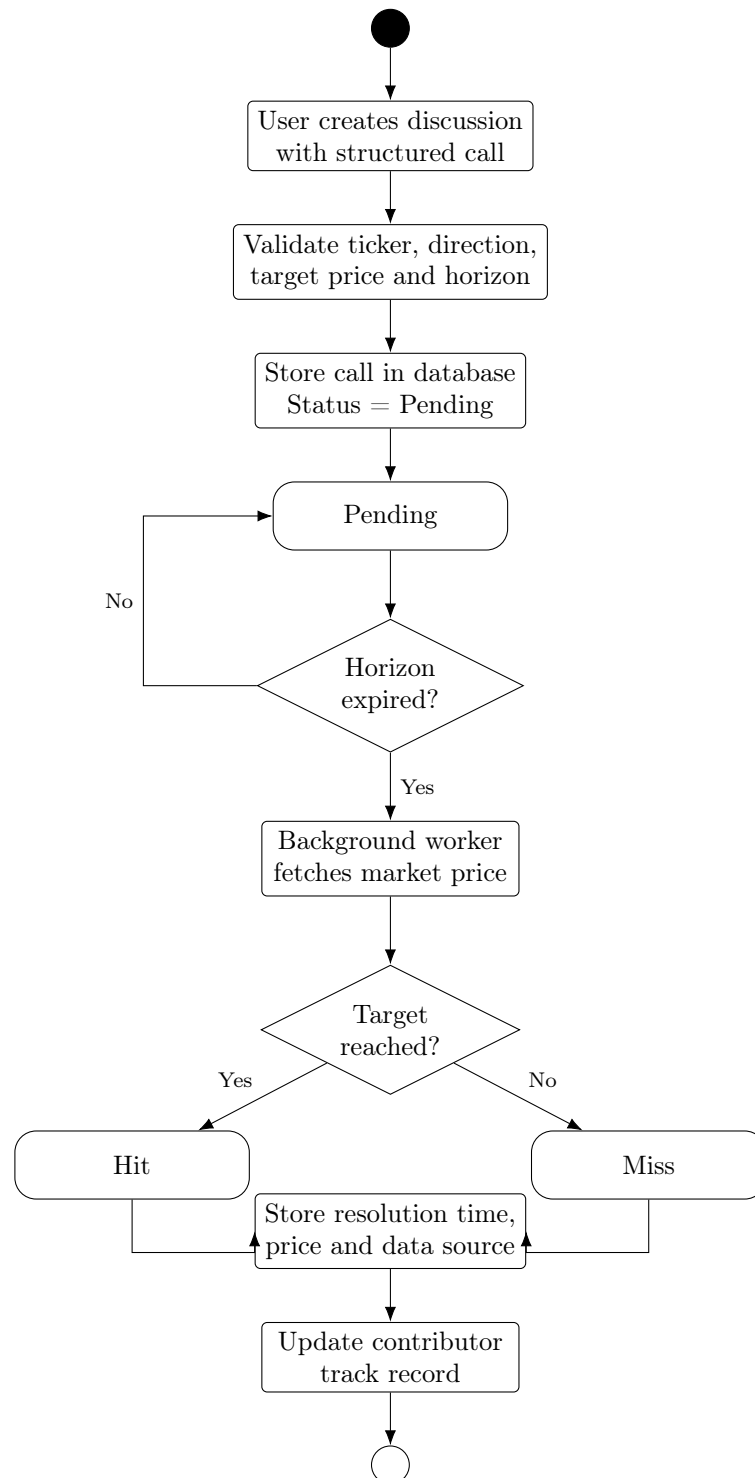


Figure 4.6: Activity diagram — creation and automatic resolution of a structured stock call.

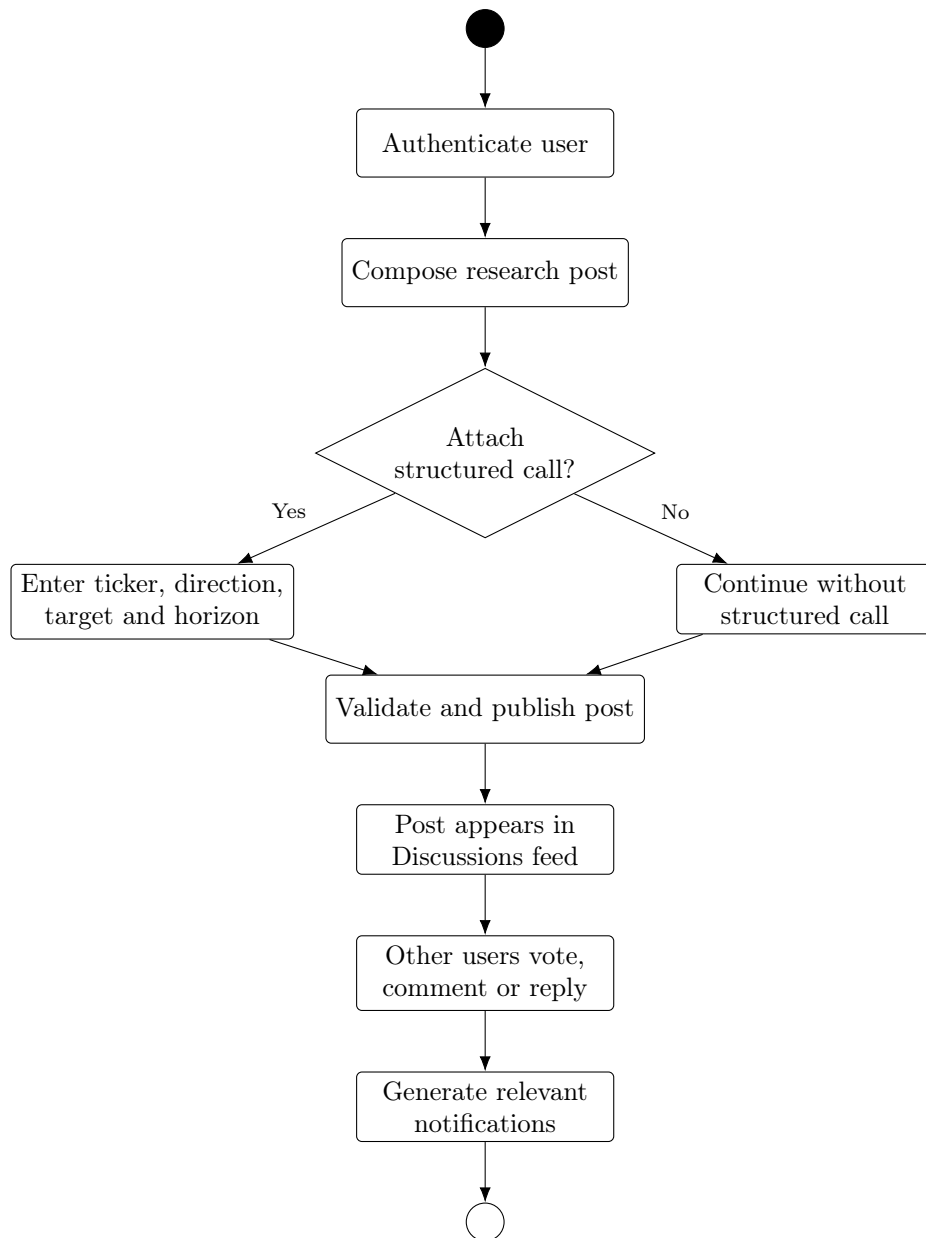


Figure 4.7: Activity diagram — publishing and interacting with a discussion.

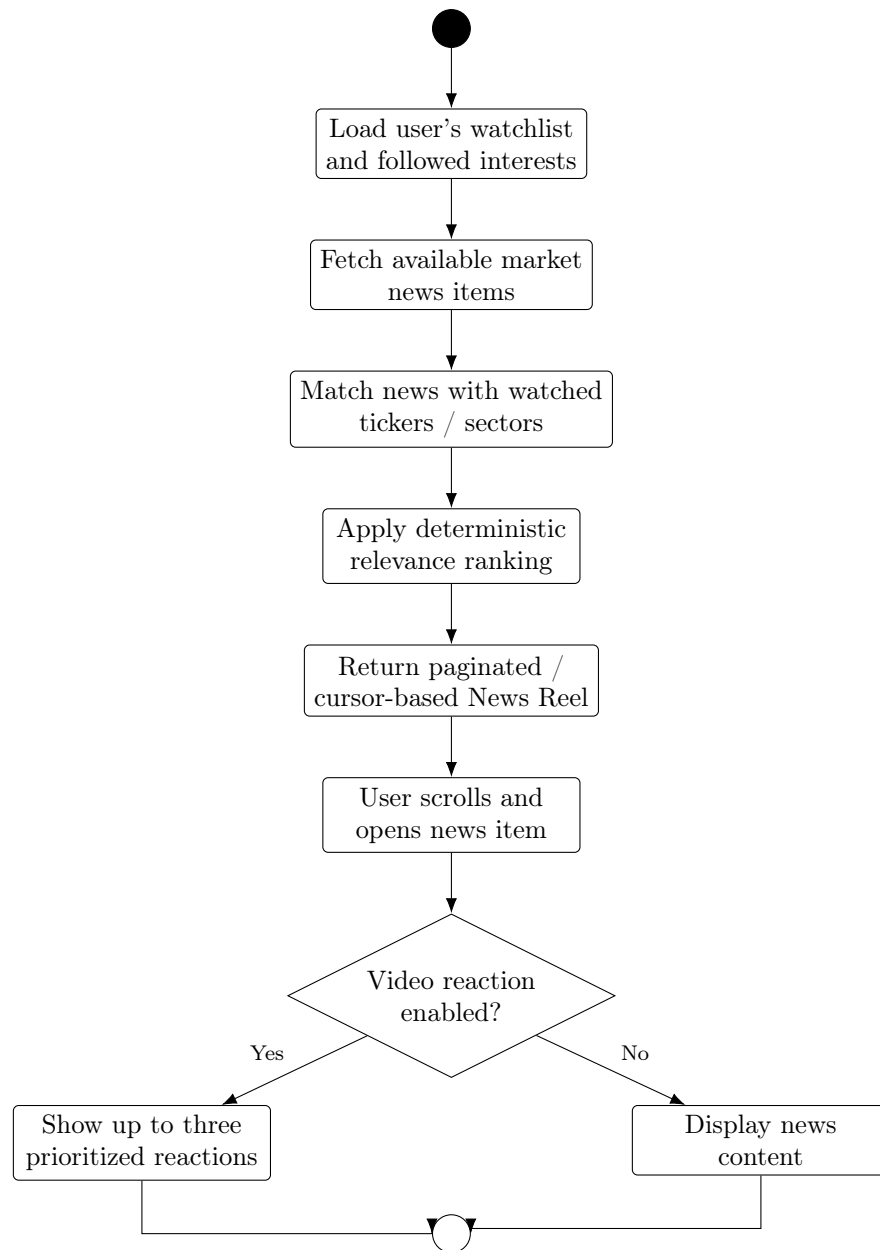


Figure 4.8: Activity diagram — generating and consuming the personalized News Reel.

4.7 Database Schema

Table 4.1: Core database tables

Table	Key Columns
users	id, username, email, password_hash, bio, created_at
follows	follower_id (FK users), followee_id (FK users), created_at
posts	id, author_id (FK users), body, created_at, sector (nullable), sentiment (nullable)
calls	id, post_id (FK posts), ticker, direction (buy/sell), target_price, horizon_date, status (open/hit/miss), resolved_at
watchlists	id, owner_id (FK users), name, is_public
watchlist_items	watchlist_id (FK), ticker
comments	id, post_id (FK), author_id (FK users), parent_comment_id (nullable), body, created_at
notifications	id, user_id (FK), type, payload_json, is_read, created_at

The accuracy score is computed, not stored redundantly: it is derived from `calls.status` grouped by author via `posts.author_id`, cached per-user in Redis, and recomputed whenever a call is resolved.

4.8 API Design (Representative Endpoints)

Table 4.2: Representative REST API endpoints

Method & Path	Purpose
POST /auth/register, /auth/login, /auth/refresh	Account creation and JWT session management
GET/PUT /users/:id	Profile read/update
GET /users/:id/accuracy	Accuracy score and call history
POST /follows/:userId, DELETE /follows/:userId	Follow / unfollow
POST /posts, GET /posts/:id	Create / read a post (with optional embedded call)
GET /feed?cursor=	Paginated, follow-based feed
POST /posts/:id/comments	Add a comment or threaded reply
POST /watchlists, PUT /watchlists/:id	Create / edit watchlist
GET /search?q=	Search users, tickers, posts
GET /notifications	List notifications for the current user

Chapter 5

UI/UX Design

5.1 Design Concept

InvestCircle's core differentiator is accountability, and the visual design is built around communicating that idea clearly. The design concept, "*Signal*", uses a calm, data-dense interface — closer to a research terminal than a typical social app — where colour is reserved almost entirely for call outcomes: teal for a hit, coral for a miss, and amber while a call is still pending. Every other surface stays neutral, so that when an outcome colour does appear, it is immediately meaningful. The signature UI element is a *track-record ribbon*: a small horizontal strip of teal/amber/coral ticks shown next to a username anywhere it appears (feed, comments, profile), giving an at-a-glance read of a contributor's track record without opening their profile.

Table 5.1: Colour palette

Name	Hex	Usage
Ink	#14181F	Primary text, nav bar, headers — the neutral anchor
Paper	#F4F6F5	App background
Signal Teal	#1FA98C	Primary action colour and "Hit" call outcome
Alert Coral	#E1583F	"Miss" call outcome, destructive actions
Amber	#E8A23D	"Pending" call status, follow/notify accents
Slate	#5B6572	Secondary text, borders, icons

Teal, coral, and amber are reserved for call status and primary actions only — never used decoratively — so a user learns to read colour as meaning within the first few sessions.

5.2 Typography

Headings use Space Grotesk (SemiBold) for a technical, data-native feel; body text uses IBM Plex Sans for legibility; tickers, prices, and timestamps use IBM Plex Mono with tabular figures enabled, so numbers stay scannable and aligned wherever they appear.

5.3 Core Components

- **Call Badge** — a pill shown wherever a call appears: filled teal "HIT +12%", filled coral "MISS -4%", or outlined amber "PENDING · 18d left".
- **Track-Record Ribbon** — ticks next to any username; tapping opens the full call history.
- **Cards** — flat, 1px-bordered, 12px-radius cards for posts, tickers, and watchlists, with no drop shadow, reinforcing the "terminal" feel.
- **Buttons** — primary actions stay neutral (Ink), since teal is reserved for outcomes; follow/notify actions use Amber; destructive actions use Coral only on the confirm state.

5.4 Screen Map

Figure 5.1 shows how the core screens connect from the app shell (bottom navigation: Feed, Discover, Watchlists, Profile).

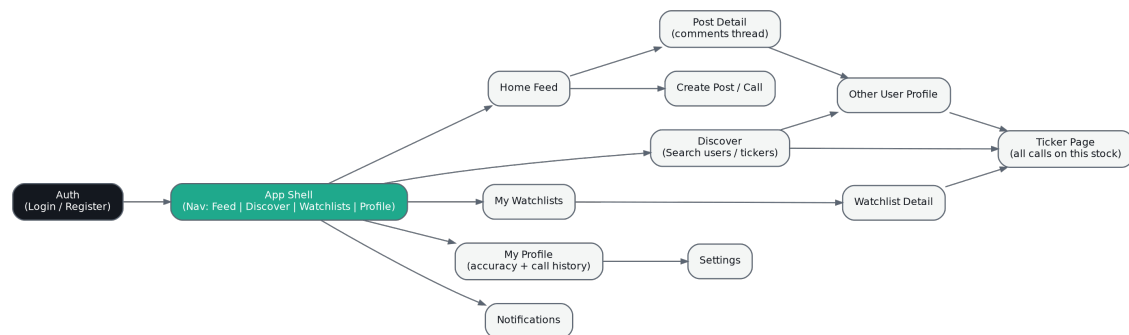


Figure 5.1: Screen map — navigation between core screens.

5.5 App Flow / Pipeline

Figure 5.2 traces the end-to-end pipeline from signup through the core accountability loop: posting a call, tracking it, automatic resolution, and the resulting update to notifications and the user's accuracy score.

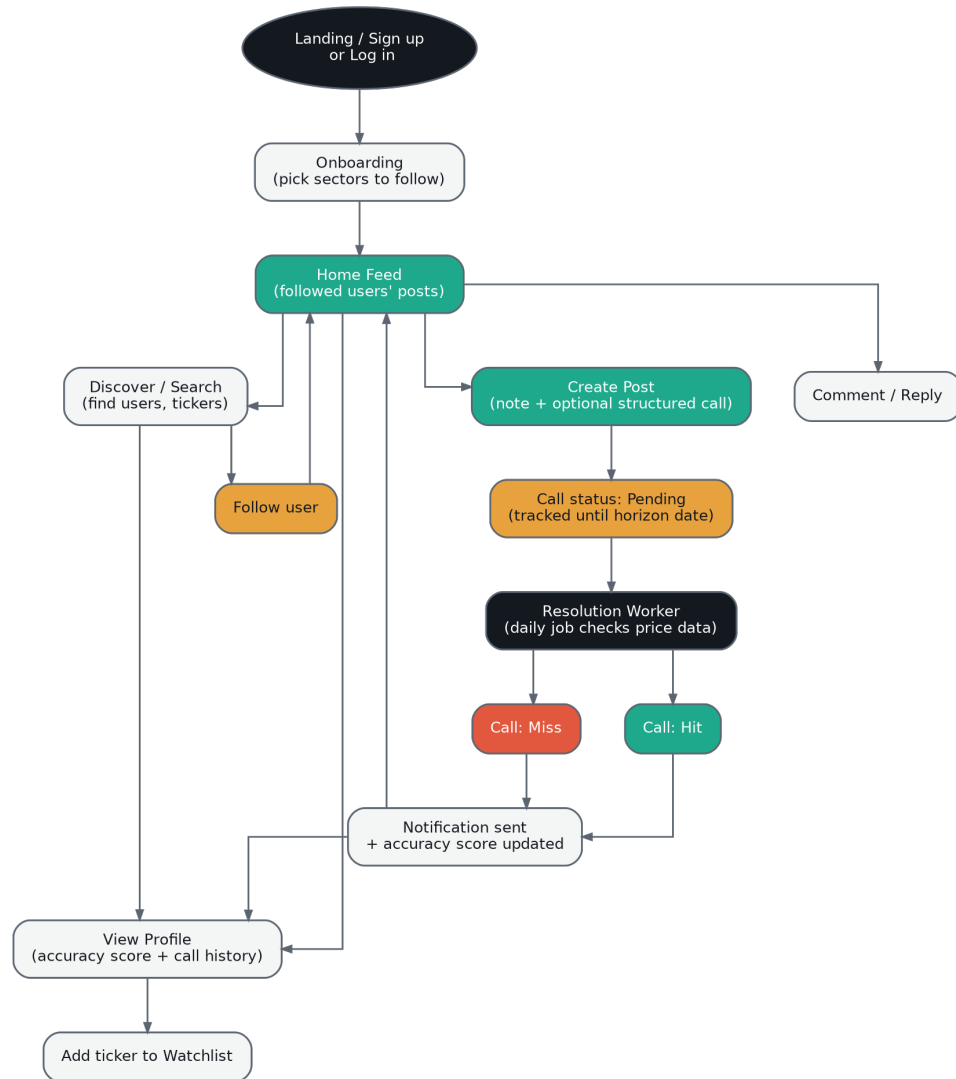


Figure 5.2: App flow — signup through call resolution and accuracy update.

5.6 Feature-to-Screen Mapping

Table 5.2 maps each screen/feature to the software engineering concept it primarily exercises and its planned implementation approach; Figure 5.3 shows the same mapping as a flow, highlighting where the external finance API enters the system (both the News/Market Blog and the Ticker Page go through a backend cron job and Redis cache — never called directly from the frontend).

Table 5.2: Screen / feature breakdown

Screen	What It Does	SE Concept	Implementation
Login / Signup	Email+password or OAuth, session handling	Authentication & security	JWT (access + refresh), bcrypt hashing
Onboarding	Pick sectors/tickers to follow	Requirements → UX flow	React form → users + preference table
Home Feed	Scrollable feed of followed users' posts	Frontend architecture, pagination	React Query, cursor-based infinite scroll
News/Market Blog	Scrolling feed of market news	3rd-party API integration	Cron job pulls a finance API, cached in Redis
Create Post/Call	Research note + optional structured call	Data modeling, validation	React form, POST /posts, server validation
Ticker Page	All calls on one stock, live price	Live data integration	Price polling, aggregation query on calls
Watchlists	Create/manage ticker lists	CRUD, relational design	Standard REST CRUD
Comments	Threaded discussion	Self-referencing FK	parent_comment_id
Profile/Accuracy	Track record, hit/miss history	Derived/computed data	Daily resolution job, score not stored redundantly
Notifications	Follow/comment/resolution alerts	Async processing	Written on trigger, polled by client
Search/Discover	Find users, tickers, posts	Query design, indexing	Postgres full-text search

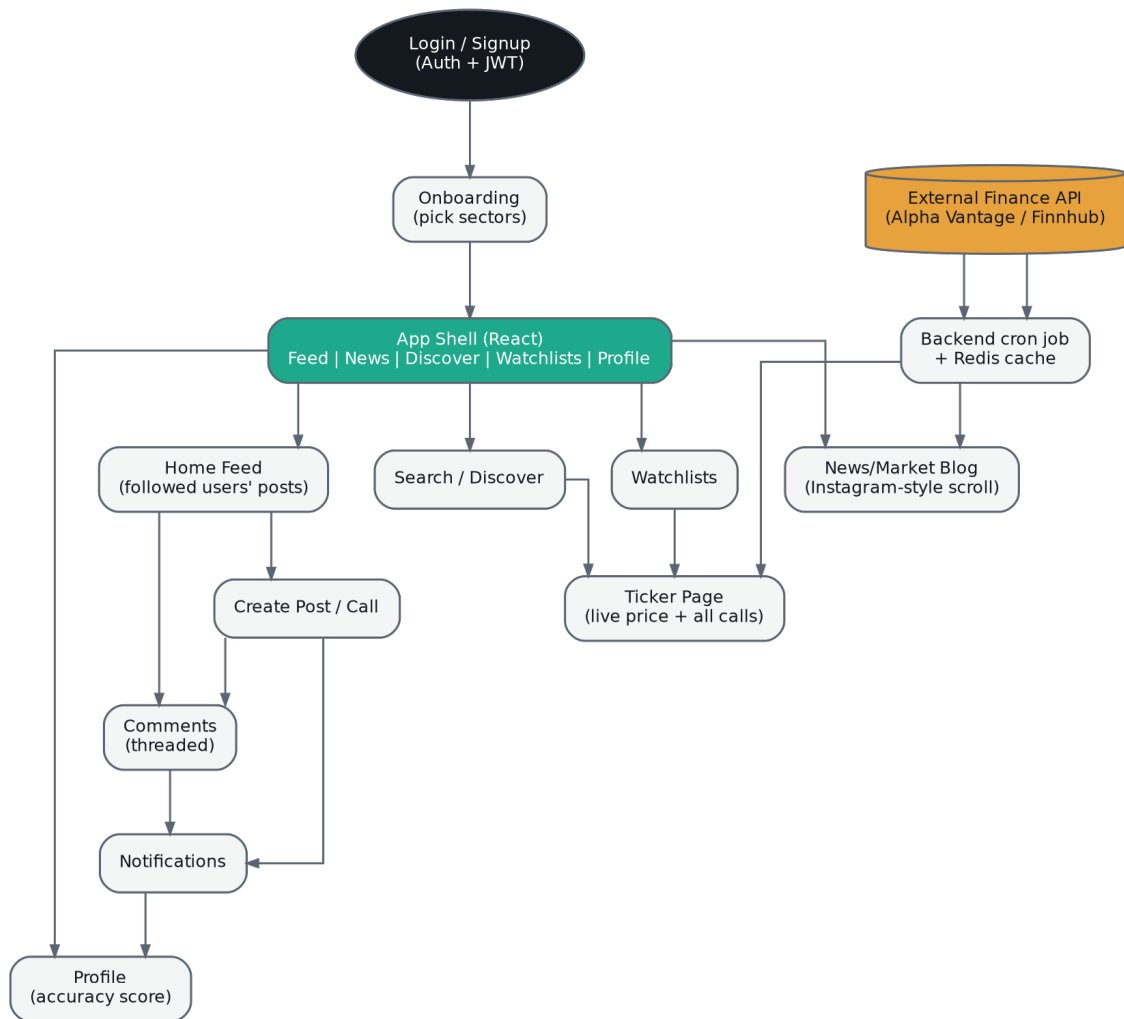


Figure 5.3: Feature flowchart — screens and external API integration points.

Chapter 6

Technology Stack

Table 6.1: Technology stack

Layer	Choice	Notes
Frontend	React + TypeScript, React Query, Tailwind	SPA, component-driven
Backend API	Node.js (Express/NestJS) or Django REST Framework	REST over GraphQL, for simplicity
Database	PostgreSQL	Strong relational fit for users/posts/follows/calls
Cache / Queue	Redis	Session cache, feed cache, background job queue (BullMQ/Celery)
AI Tagging Service	Python microservice (FastAPI) wrapping a small transformer model	Isolated, called async via queue
Auth	JWT (access + refresh tokens)	Stateless, scalable
Deployment	Docker Compose (dev), single cloud VM or free-tier PaaS (prod demo)	Simple infra for a student project
CI	GitHub Actions	Lint + unit tests on every PR

Chapter 7

Work Completed So Far

This chapter summarises the work completed at the prototype stage, defined in the project plan (Section 7.5).

7.1 Requirement and Design Documentation

- Project proposal, Product Requirements Document (PRD), and Technical Requirements Document (TRD) finalised, covering scope, functional/non-functional requirements, database schema, API design.
- UI/UX design document finalised: visual design system ("Signal"), colour palette, typography, core components, screen map, and app flow (Chapter 5).
- Screen-and-feature breakdown document finalised, mapping every screen to the software-engineering concept it exercises and its planned implementation.

7.2 System Design and Diagrams

- Use case diagram covering all actors (User, Administrator, and the automated/external News Provider, Market Data Provider, and AI Service actors) and their use cases, including «include» relationships.
- Class diagram of the core domain model (User, Post, Call, Comment, Follow, Watchlist, WatchlistItem, Notification).
- Sequence diagram of the create-post-with-call flow, showing the asynchronous AI tagging path.
- Three activity diagrams: structured call resolution, the discussion workflow, and the personalised News Reel.
- System architecture and deployment diagrams.

All diagrams are reproduced in Chapter 4.

7.3 Seed and Test Data

Since InvestCircle’s schema holds entirely user-generated content that does not exist yet, and no real user data can ethically be used for a class project, a synthetic seed/test dataset was generated to support development and demonstration:

- **Real reference data:** 34 NSE Nifty-50 tickers with company name and sector classification (BFSI, IT, FMCG, Auto, Healthcare, Oil & Gas, Metals & Mining, Power, Telecom, Consumer Durables, Construction & Infrastructure) — the only real-world data in the seed set.
- **Synthetic data:** 60 users, a ~470-edge follow graph, 150 research posts (templated, ticker-linked), 92 structured calls with realistic hit/miss/open distributions, 220 threaded comments, 129 watchlists with 841 watchlist items, and 478 derived notification events — all generated with a fixed random seed for reproducibility.
- **AI tagging sample set:** a separate, balanced set of 240 labelled samples (80 positive / 80 neutral / 80 negative, all 34 tickers covered), purpose-built for testing or lightly training the sector/sentiment tagging microservice independent of the seed posts above.
- A full PostgreSQL seed script (schema + inserts, matching Section 4.7) and equivalent per-table CSVs were generated for direct use by whichever backend framework the team settles on.

7.4 Real Historical Price Data

Since the call-resolution workflow (Section 4.7, Figure 4.6) depends on real end-of-day price data, two supporting scripts were prepared, and the real-data path has since been exercised end to end:

- A script to merge a downloaded historical NSE price dataset into the synthetic `calls` table, for a reproducible demo without any live API dependency.
- A production-oriented worker script using the `yfinance` library to fetch live closing prices for NSE tickers and resolve open calls on a schedule, mirroring the call-resolution worker described in Figure 4.6.
- **A real-data backtest set** of 200 structured calls (`backtest_calls_real.csv`) was generated directly from actual NSE trading history rather than merged after the fact: for each call, a real historical trading day was chosen first, the target price was computed off the *real* closing price on that day, and the hit/miss outcome was computed against the *real* closing price on the horizon date. This produced a 20% hit rate (40/200) — an unforced number, simply what falls out of random real entry points and directions against real price history — and gives the team a concrete, independently verifiable proof that the resolution logic in Figure 4.6 produces correct outcomes against real market data, not just synthetic numbers.

7.4.1 Data Provenance

To be precise about what in this project is real versus synthetic, since this distinction matters for evaluation:

- **Real:** the 32-ticker daily OHLCV price history used for the backtest (source: a public mirror of the NSE Nifty-50 historical dataset, January 2000 – April 2021 coverage; see References). Every price and date in `stock_market/*.csv` is unmodified real market data. Three of the project's 34 reference tickers — `APOLLOHOSP`, `HDFCLIFE`, `SBILIFE` — are not present in this dataset, as they were not Nifty-50 constituents as of its 2021 cutoff.
- **Synthetic, grounded in real data:** the 200 backtest calls themselves. No public dataset of "structured stock calls with tracked outcomes" exists — this is InvestCircle's own product concept — so the calls were generated, but every price they are judged against is real, as described above.
- **Fully synthetic:** the seed/test dataset in Section 4.7 above (users, follow graph, posts, comments, watchlists, notifications, and the AI tagging sample set), used to populate a realistic-looking *current* (2024–2026) instance of the app for demoing screens end to end. This is kept separate from the real-data backtest set, which is deliberately historical (2000–2021) and used only to validate the resolution logic, not to populate the live-looking demo.

7.4.2 Preprocessing Applied to the Real Dataset

The raw historical dataset required several preprocessing steps before it could be used:

1. **Decompression** — the source was distributed as a compressed archive and had to be extracted.
2. **Ticker filtering** — the raw dataset covers 50 stocks; filtered down to the 32 that intersect with the project's reference ticker list.
3. **Type conversion** — dates and prices are stored as text in the raw CSVs; parsed into proper date and floating-point types before any comparison logic was applied.
4. **Trading-day lookup** — a call's horizon date is not guaranteed to fall on an actual trading day (weekends/holidays), so resolution uses a "closing price on or after this date" lookahead rather than a naive exact-date match.
5. **Column selection** — the raw data carries 14 columns per row (including `Turnover`, `Trades`, `Deliverable Volume`); only `Close` was required for the resolution logic above, so the remaining columns are retained in the files but unused.

The raw dataset has known gaps in the `Trades` and `Deliverable Volume` columns for some early-2000s rows; this did not affect the work above since those columns were not used, but is noted here for transparency.

7.5 Milestone Plan

Table 7.1: Milestone plan

S.No.	Milestone
1	Requirements sign-off, system design, DB schema, API contract, repo/CI setup
2	Auth, user profiles, and watchlists — backend + frontend, deployed to staging
3	Posts, structured calls, and feed (follow-based ranking)
4	Comments, notifications, search; call-resolution background job
5	Accuracy tracking/profile stats, AI sector/sentiment tagging bolt-on, moderation basics
6	Performance/load testing, bug-fixing, deployment hardening, documentation, final demo prep

The work summarised in this chapter corresponds to requirements sign-off and system design together with early groundwork -seed data and price-resolution tooling deliverables depend on.

Chapter 8

Testing Strategy

- **Unit tests** for business logic (accuracy calculation, call resolution logic) — target 70%+ coverage.
- **Integration tests** for core API endpoints (auth, posts, follow, feed).
- **Seeded test dataset** (Chapter 7) — synthetic users, posts, and resolved/open calls — for demo and load testing.
- **Manual QA pass** on core user flows before each monthly milestone review.

8.1 Security Considerations

- Passwords hashed with bcrypt/Argon2; never logged.
- JWT access tokens short-lived (~15 min), refresh tokens rotated and stored hashed.
- Input validation and parameterised queries (via ORM) on all endpoints to prevent SQL injection.
- Rate limiting on authentication and post-creation endpoints.
- CORS restricted to the deployed frontend origin.

Chapter 9

Conclusion and Future Work

At the prototype stage, InvestCircle’s requirement specification and system design are complete: the functional and non-functional requirements are finalised, the database schema and API surface are defined, and the use case, class, sequence, activity, architecture, and deployment views described in Chapter 4 give a consistent, implementable picture of the system. The UI/UX design system and screen flows (Chapter 5) translate those requirements into a concrete, distinctive interface built around the platform’s central idea — accountability. Supporting seed data, a labelled dataset for the AI tagging feature, and tooling to resolve calls against real historical price data (Chapter 7) are in place ahead of implementation, so that development, testing, and the first demo do not block on data availability.

9.1 Future Work

- Implement the backend (auth, posts, calls, feed, follow graph) and the call-resolution and notification workers.
- Implement the frontend screens per the UI/UX design system and screen map.
- Wire in the AI sector/sentiment tagging microservice using the labelled sample dataset already prepared, keeping it strictly optional and off the critical request path.
- Integrate a live market-data provider for the News Reel and call-resolution worker , building on the price-fetching tooling already prepared.
- Load-test the feed and profile endpoints against the seeded dataset and harden deployment ahead of the final demo.

References

1. National Stock Exchange of India (NSE) — Nifty 50 index constituents and sector classification, <https://www.nseindia.com/>.
2. Rao, R. — *NIFTY-50 Stock Market Data (2000–2021)*, Kaggle Datasets, <https://www.kaggle.com/datasets/rohanrao/nifty50-stock-market-data>.
3. Yuvraj Singh — GitHub mirror of the above dataset (used directly in this project, as Kaggle requires an account to download): <https://github.com/Yuvraj-Singh-Bits-Pilani/Analysis-of-Nifty50>.
4. Periwal, N. — *NSE India Stock Data (1990–2021)*, Kaggle Datasets, <https://www.kaggle.com/datasets/stoicstatic/india-stock-data-nse-1990-2020>.
5. `yfinance` — Python library for Yahoo Finance market data, <https://pypi.org/project/yfinance/>.
6. OWASP Foundation — Password storage and authentication best practices (bcrypt/Argon2, JWT), <https://owasp.org/>.